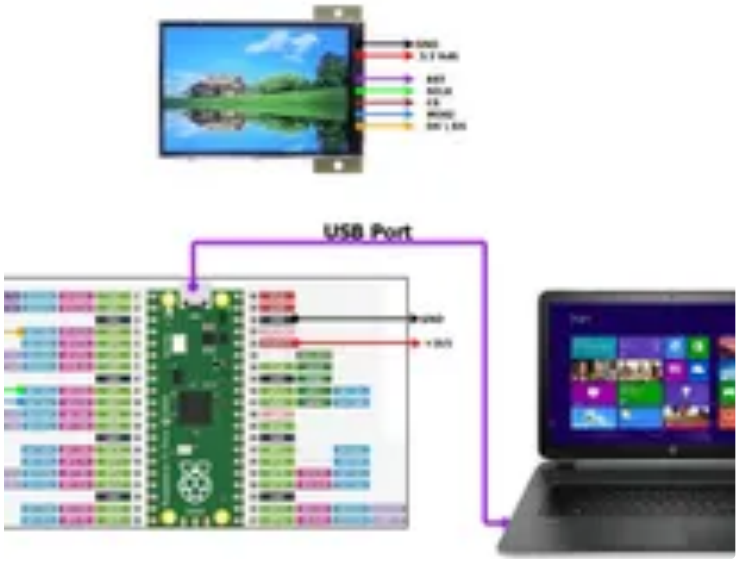
RPi Pico – 3.5 Inch (320x480) HVGA TFT LCD (ILI9488) – Bitmap Image Photo Frame – Internal Flash

By PugazhM in Circuits > Raspberry Pi 23,219 9 16

BY-NC-SA



The photo frame experimentation is about interfacing 320x480 HVGA TFT LCD with Raspberry Pi Pico. This experiment displays, internally stored images on TFT LCD screen. It uses python program for implementing file read, bitmap image handling functionalities.

Video link

VESZLE - The Art Of Electronics

Watch on

Step 1: Abstract

The Raspberry Pi Pico is a tiny, fast, and versatile board built using RP2040 features a dual-core Arm Cortex-M0+ processor with 264KB internal RAM and support for up to 16MB of off-chip Flash. It provides wide range of flexible I/O options includes I2C, SPI, and uniquely Programmable I/O (GPIO) pins.

Nowadays beautiful TFT LCD screens are getting cheaper and using it in an embedded design, makes it more user friendly. In this instruct-able, explains about connecting the 320x480, 3.5Inch TFT LCD, with ILI9488 driver and SPI interfacing into Raspberry Pi Pico. The TFT LCD can be connected to the Raspberry Pi Pico SPI bus. It needs minimum number of port pins (4).

The photo frame experimentation is about interfacing 320x480 TFT LCD with Raspberry Pi Pico. This experiment displays, internally stored images on TFT screen. It uses python program for implementing file read, bitmap image handling functionalities.

Step 2: Reference

“Raspberry Pi Pico, Getting Started on Board Blink LED” Instruct-able / You-tube by PugazhM

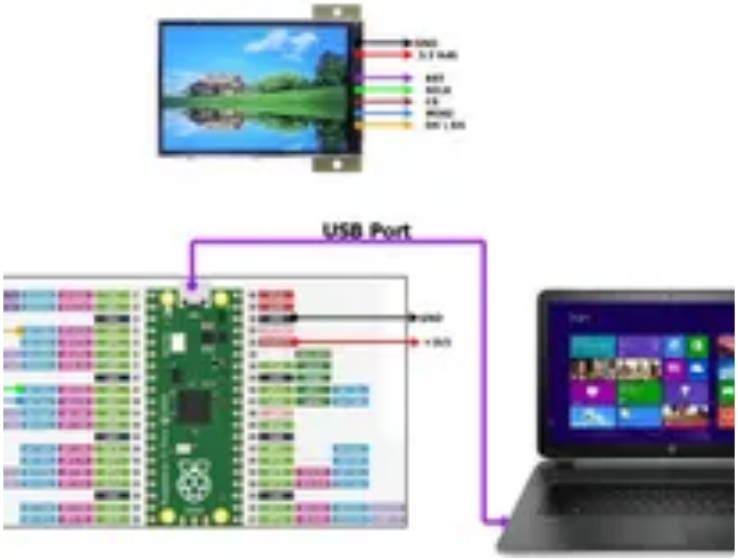
Step 3: Components

Raspberry Pi Pico

Micro USB Cable

320x480, ILI9488 SPI TFT LCD

Step 4: Schematic



The TFT LCD (3.5 inch, 320x480 pixel, ILI9488 LCD controller), is used for this instructable.

The LCD is easily interfaced with RPi Pico SPI bus, and it needs minimum of four Digital IO lines.

The ILI9488 LCD Controller is a 16.7M single-chip SoC driver for a-Si TFT liquid crystal display panels with a resolution of 320(RGB) x 480 dots.

The ILI9488 is comprised of a 960-channel source driver, a 480-channel gate driver, 345,600 bytes of on chip GRAM for graphic data of 320 (RGB) (H) x 480 (V) x 18 dots.

The LCD operates at 3.3 Volt Logic.

The ILI9488 supports 8-colors display and sleep mode power management functions, ideal for portable products where battery power conservation is desirable, such as digital cellular phones, smart phones, MP3 players, personal media players and similar devices with color graphics displays.

RPi Pico SPI port is connected to the LCD (GPIO6 - SCLK, and GPIO7 – MOSI).

RPi Pico GPIO pin GPIO0, GPIO1 and GPIO2 are connected to CS, RST and DC\RS pin of TFT LCD.

Step 5: Bitmap Header and Image File Manipulation

All of the integer values in a bitmap file are stored in little-endian format (i.e., least-significant byte first) The first 2 bytes of the BMP file format are the character "B" then the character "M" in ASCII encoding.

Next 4 bytes (0x02 to 0x05) BMP file providing the file size. Next 4 bytes (0x05 to 0x09) BMP file are reserved.

Next 4 bytes (0x0A to 0x0D) BMP file providing the offset, i.e., starting address, of the byte where the bitmap image data (pixel array) can be found. Next 4 bytes (0x0E to 0x11) BMP file provides size of the header.

Next 2 bytes (0x12 to 0x13) BMP file provides width of the BMP image, in pixels. Next 2 bytes (0x14 to 0x15) BMP file provides height of the BMP image, in pixels.

Next 2 bytes (0x16 to 0x17) BMP file provides number of color planes.

Next 2 bytes (0x18 to 0x19) BMP file provides number of bits per pixel. Since the 320x480 (ILI9488) TFT supports RGB565, and flash memory storage is limited, then one has to convert the BMP files into RG565 (16-bit format), and re-size the image to 480 width, 320 height or smaller

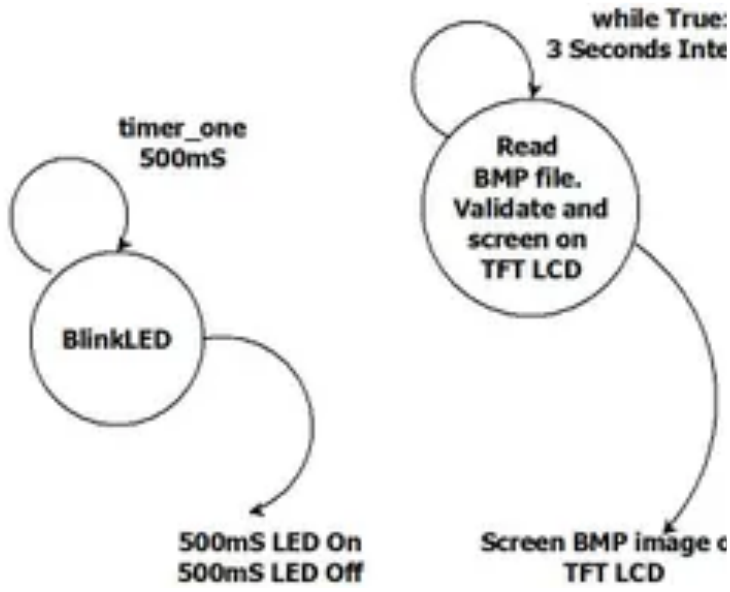
The 480x320 image, approximate BMP file size is 300KB. (480x320x2 = 307200 bytes + header size)

RPi Pico provides 2MB of storage space. MicroPython uses, around 600KB. 1.4 MB flash memory is available storing the bitmap files.

The GIMP is free, and open-source raster graphics editor for image manipulation and editing.

Open an image file into GIMP. Edit and resize the image into 480x320 or lesser. Export the file into RGB565 BMP format. File -- Export as -- image.bmp -- Advanced option -- 16 Bits -- R5 G6 B5

Step 6: Python Program – 320x480 TFT, Bitmap File Handling



Open the RPi Pico as drive, and then copy the bitmap files into root directory.

The timer_one is initialized and callbacks the "BlinkLED" functionality for toggling on board LED at 500mS duration. (frequency = 1)

The TFT LCD class provides basic firmware functionalities like Init, ResetDevice, WriteDevice, WriteDataToDevice, WriteBlock and FillRectangle.

The python program, initializes ILI9433 TFT LCD with SPI interface.

The python program reads the bitmap files, and screens one after another at 3 seconds interval.

```
'''
Demonstrates the use of 320x480 HVGA TFT (ILI9488) TFT, initialization and internal flash ph
Draw Bitmap files (BMP) on screen

* The Raspberry Pi Pico pin connections for TFT is given below:
# TFT Power Pins
* TFT VCC pin to 3V3
* TFT GND pin to GND

# TFT SPI Pins
* TFT SCLK pin to GPIO6
* TFT MOSI pin to GPIO7
* TFT CS pin to GPIO0

# TFT Control Pins
* TFT RST pin to GPI01
* TFT RS\DC pin to GPIO2

 Name:- M.Pugazhendi
 Date:-  24thAug2021
```

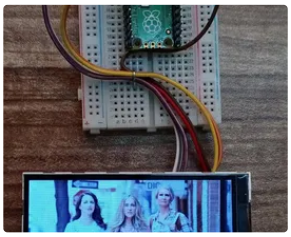
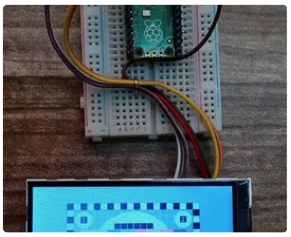
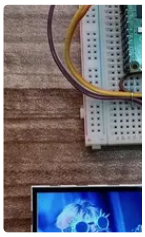
```
 Version:- V0.1
 e-mail:- muthuswamy.pugazhendi@gmail.com
'''

from micropython import const
from machine import Pin, SPI
import time, os, ustruct

READ_DISPLAY = const(0x0f);SLEEP_OUT = const(0x11);GAMMA_SET = const(0x26);DISPLAY_ON = cons
COLUMN_ADDRESS_SET = const(0x2a);PAGE_ADDRESS_SET = const(0x2b);RAM_WRITE = const(0x2c);RAM_
MEMORY_ACCESS_CONTROL = const(0x36);VER_SCROLL_ADDRESS = const(0x37);NEG_GAMMA_CONTROL = con
PIXEL_FORMAT_SET = const(0x3a);POWER_CONTROL_A = const(0xcb);POWER_CONTROL_B = const(0xcf)
DRIVER_TIMING_CONTROL_A = const(0xe8);DRIVER_TIMING_CONTROL_B = const(0xea);POWER_ON_CONTROL
PUMP_RATIO_CONTROL = const(0xf7);POWER_CONTROL_1 = const(0xc0);POWER_CONTROL_2 = const(0xc1)
VCOM_CONTROL_1 = const(0xc5);VCOM_CONTROL_2 = const(0xc7);FRAME_RATE_CONTROL = const(0xb2)
DISPLAY_FUNCTION_CONTROL = const(0xb6);ENABLE_3G = const(0xf2);POS_GAMMA_CONTROL = const(0xe

MEMORY_BUFFER = const(1024) # SPI Write Buffer

class ILI9488:
    def __init__(self, spi, cs, dc, rst, w, h, r):
        self.spi = spi;self.cs = cs;self.dc = dc;self.rst = rst
        self.init_width = w;self.init_height = h
        self.width = w;self.height = h;self.rotation = r
        self.cs.init(self.cs.OUT, value=1);self.dc.init(self.dc.OUT, value=0)
        self.rst.init(self.rst.OUT, value=0)
        self.ResetDevice()
        self.Init()
        self.buffer = bytearray(MEMORY_BUFFER * 2)
        self.color_map = bytearray(b'\x00\x00\xFF\xFF') #default white foregraound, black ba
        self.screen_x = 0;self.screen_y = 0


    def SetPosition(self,x,y):
        self.screen_x,self.screen_y = x,y

    def Init(self):
        for command, data in (
            (READ_DISPLAY, b"\x03\x80\x02"),(POWER_CONTROL_B, b"\x00\xc1\x30"),
            (POWER_ON_CONTROL, b"\x64\x03\x12\x81"),(DRIVER_TIMING_CONTROL_A, b"\x85\x00\x78
            (POWER_CONTROL_A, b"\x39\x2c\x00\x34\x02"),(PUMP_RATIO_CONTROL, b"\x20"),
            (DRIVER_TIMING_CONTROL_B, b"\x00\x00"),(POWER_CONTROL_1, b"\x23"),
            (POWER_CONTROL_2, b"\x10"),(VCOM_CONTROL_1, b"\x3e\x28"),(VCOM_CONTROL_2, b"\x86
            self.WriteDevice(command, data)

        if self.rotation == 0:                  # 0 deg
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\x48")
            self.width,self.height = self.init_height,self.init_width
        elif self.rotation == 1:                # 90 deg
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\x28")
            self.width,self.height = self.init_width,self.init_height
        elif self.rotation == 2:                # 180 deg
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\x88")
            self.width,self.height = self.init_height,self.init_width
        elif self.rotation == 3:                # 270 deg
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\xE8")
```

```
            self.width,self.height = self.init_width,self.init_height
        elif self.rotation == 4:                # Mirrored + 0 deg
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\xC8")
            self.width,self.height = self.init_height,self.init_width
        elif self.rotation == 5:                # Mirrored + 90 deg
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\x68")
            self.width,self.height = self.init_width,self.init_height
        elif self.rotation == 6:                # Mirrored + 180 deg
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\x08")
            self.width,self.height = self.init_height,self.init_width
        elif self.rotation == 7:                # Mirrored + 270 deg
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\xA8")
            self.width,self.height = self.init_width,self.init_height
        else:
            self.WriteDevice(MEMORY_ACCESS_CONTROL, b"\x08")

        for command, data in (
            (PIXEL_FORMAT_SET, b"\x55"),(FRAME_RATE_CONTROL, b"\x00\x18"),
            (DISPLAY_FUNCTION_CONTROL, b"\x02\x02\x3B"),(ENABLE_3G, b"\x00"),(GAMMA_SET, b"\
            (POS_GAMMA_CONTROL, b"\x0f\x31\x2b\x0c\x0e\x08\x4e\xf1\x37\x07\x10\x03\x0e\x09\x
            (NEG_GAMMA_CONTROL, b"\x00\x0e\x14\x03\x11\x07\x31\xc1\x48\x08\x0f\x0c\x31\x36\x
            self.WriteDevice(command, data)
        self.WriteDevice(SLEEP_OUT)
        time.sleep_ms(120)
        self.WriteDevice(DISPLAY_ON)

    def ResetDevice(self):
        self.rst(0);time.sleep_ms(50);self.rst(1);time.sleep_ms(50)

    def WriteDevice(self, command, data=None):
        self.dc(0)
        self.cs(0)
        self.spi.write(bytearray([command]))
        self.cs(1)
        if data is not None:
            self.WriteDataToDevice(data)

    def WriteDataToDevice(self, data):
        self.dc(1);self.cs(0);self.spi.write(data);self.cs(1)

    def WriteBlock(self, x0, y0, x1, y1, data=None):
        self.WriteDevice(COLUMN_ADDRESS_SET, ustruct.pack(">HH", x0, x1))
        self.WriteDevice(PAGE_ADDRESS_SET, ustruct.pack(">HH", y0, y1))
        self.WriteDevice(RAM_WRITE, data)

    def FillRectangle(self, x, y, w, h, color=None):
        x = min(self.width - 1, max(0, x));y = min(self.height - 1, max(0, y))
        w = min(self.width - x, max(1, w));h = min(self.height - y, max(1, h))
        if color:
            color = ustruct.pack(">H", color)
        else:
            color = self.color_map[0:2] #background
        for i in range(MEMORY_BUFFER):
            self.buffer[2*i]=color[0]; self.buffer[2*i+1]=color[1]
        chunks, rest = divmod(w * h, MEMORY_BUFFER)
```

```
        self.WriteBlock(x, y, x + w - 1, y + h - 1, None)
        if chunks:
            for count in range(chunks):
                self.WriteDataToDevice(self.buffer)
        if rest != 0:
            mv = memoryview(self.buffer)
            self.WriteDataToDevice(mv[:rest*2])


#Initialize the onboard LED as output
led = machine.Pin(25,machine.Pin.OUT)

# Toggle LED funtionality
def BlinkLED(timer_one):
    led.toggle()

# https://forum.micropython.org/viewtopic.php?t=1420 Roberthh
@micropython.asm_thumb
def reverse(r0, r1):               # bytearray, len(bytearray)
    b(loopend)
    label(loopstart)
    ldrb(r2, [r0, 0])
    ldrb(r3, [r0, 1])
    strb(r3, [r0, 0])
    strb(r2, [r0, 1])
    add(r0, 2)
    label(loopend)
    sub (r1, 2)  # End of loop?
    bpl(loopstart)

SCR_WIDTH,SCR_HEIGHT,SCR_ROT = const(480),const(320),const(5)

TFT_CLK_PIN,TFT_MOSI_PIN,TFT_MISO_PIN,TFT_CS_PIN = const(6),const(7),const(4),const(0)
TFT_RST_PIN,TFT_DC_PIN = const(1),const(2)

spi = SPI(0,baudrate=40000000,miso=Pin(TFT_MISO_PIN),mosi=Pin(TFT_MOSI_PIN),sck=Pin(TFT_CLK_
display = ILI9488(spi,cs=Pin(TFT_CS_PIN),dc=Pin(TFT_DC_PIN),rst=Pin(TFT_RST_PIN),w=SCR_WIDTH

display.SetPosition(0,0);display.FillRectangle(0,0,480,320,0xBDF7)

# Read files.
bitmap_image_files = os.listdir("/")

#Initialize timer_one. Used for toggling the on board LED
timer_one = machine.Timer()
#Timer one initialization for on board blinking LED at 500mS interval
timer_one.init(freq=1, mode=machine.Timer.PERIODIC, callback=BlinkLED)

# Opens bitmap file. Displays the file @ 3 Sec interval
while True:
    for x in range(len(bitmap_image_files)):
        # Open file
        f = open(bitmap_image_files[x],'rb')


        # Check if it bitmap file
        if f.read(2) == b'BM':  #header
            dummy = f.read(8) #file size(4), creator bytes(4)
```

```
        offset = int.from_bytes(f.read(4), 'little')
        hdrsize = int.from_bytes(f.read(4), 'little')
        width = int.from_bytes(f.read(4), 'little')
        height = int.from_bytes(f.read(4), 'little')
        MEM_SIZE = 5*1024
        x = (SCR_WIDTH / 2) - (width / 2)
        y = (SCR_HEIGHT / 2) - (height / 2)

        if((width*height*2)>MEM_SIZE):
            no_of_read_buffer,balance_memory_size = (width*height*2)/MEM_SIZE, (width*he
        else:
            no_of_read_buffer = 0
            balance_memory_size = (width*height*2)
        x = int(x)
        y = int(y)
        no_of_read_buffer = int(no_of_read_buffer)
        balance_memory_size = int(balance_memory_size)

        if(no_of_read_buffer == 0):
            display.WriteBlock(x,y,x+width - 1, y+height - 1,None)
            dummy = f.seek(offset)
            buf = f.read(width*height*2)
            reverse(buf,width*height*2)
            display.WriteDataToDevice(buf)
            time.sleep(5)
        else:
            if(width%2 == 0x00):
                display.WriteBlock(x,y,x+width-1, y+height - 1,None)
            else:
                display.WriteBlock(x,y,x+width, y+height - 1,None)

            for i in range(no_of_read_buffer):
                p = offset + (i*MEM_SIZE)
                dummy = f.seek(p)
                buf = f.read(MEM_SIZE)
                reverse(buf,MEM_SIZE)
                display.WriteDataToDevice(buf)
            p = offset + (no_of_read_buffer*MEM_SIZE)
            dummy = f.seek(p)
            buf = f.read(balance_memory_size)
            reverse(buf,balance_memory_size)
            display.WriteDataToDevice(buf)
            time.sleep(3)

    # Close file
    f.close()
    display.FillRectangle(0,0,480,320,0xBDF7)
```

Step 7: Conclusion

The project is successfully completed with Raspberry Pi Pico and 320x480

TFT LCD shield 320x480 HVGA shield represent flexible in expensive display technology and can be used for many embedded projects as mixed image and alpha numerical display. It consumes less power.

Step 8: Video Link

Result_Video.mp4

Visit "VESZLE - The Art Of Electronics" you tube channel for further information and videos.

https://www.youtube.com/channel/UCY4mekPLfeFinbQH...

If you enjoyed this instruct-able, then make sure you subscribe

The 320x480 TFT LCD Phot frame video link

https://youtu.be/Nv5WN5LceFg